



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT)

Academic Year 2024 - 2025

SAP ID: 60019220125

Name: Gargi Sukhatankar

SAP ID:60019220125

Branch: CSE (ICB)

Batch: B1

Subject: CCL

Aim: To implement Database as a Service on SQL/NoSQL databases Firebase/Supabase Theory:

Database as a Service (DBaaS)

Database as a Service (DBaaS) is a cloud-based service model where the database management system (DBMS) is provided as a service by a cloud provider. DBaaS eliminates the need for organizations to manage the hardware, software, and the administration of the database system. Instead, the cloud provider handles everything from infrastructure provisioning to database backups and scaling, allowing businesses to focus on their core applications and services. DBaaS supports both SQL and NoSQL databases, each catering to different application needs.

SQL and NoSQL Databases in DBaaS

- 1. SQL Databases:** SQL databases (relational) use Structured Query Language (SQL) and are ideal for applications with structured data and complex queries. DBaaS for SQL databases provides automatic scaling, high availability, and backup services.
 - **Example Providers:** Google Cloud SQL, AWS RDS.
- 2. NoSQL Databases:** NoSQL databases are designed for handling unstructured or semistructured data. These databases are more flexible and can scale horizontally, making them suitable for real-time analytics and large datasets. ◦ **Example Providers:** Google Firestore, AWS DynamoDB.

Firebase as DBaaS

Firebase is a platform provided by Google for building mobile and web applications. It includes a fully managed, scalable NoSQL cloud database known as **Cloud Firestore**. Firebase also supports real-time databases for applications requiring low-latency and high-concurrency.

- **Features of Firebase:**

- **Real-time Database:** Firebase's Realtime Database allows data to be synchronized across all clients in real-time, which is essential for applications like chat or live updates.
- **Cloud Firestore:** Firestore is a flexible, scalable NoSQL cloud database designed for storing and syncing data for client- and server-side apps. It offers hierarchical data structures and supports complex queries.

Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT) Academic

Year 2024 - 2025

SAP ID: 60019220028



- **Authentication:** Firebase includes integrated authentication services, allowing apps to handle user authentication securely.
- **Offline Support:** Firestore and the Realtime Database provide offline support, enabling apps to function without an internet connection.
- **Serverless:** Firebase is serverless, meaning users do not need to manage the infrastructure or scale the database manually.
- **Implementation:** Firebase provides SDKs for multiple platforms, including Android, iOS, and the web. Developers can use Firebase to manage real-time databases, user authentication, and file storage without managing servers.

Supabase as DBaaS

Supabase is an open-source alternative to Firebase that provides backend services, including SQL databases, authentication, and storage. It is built on top of PostgreSQL, a powerful relational database management system. Supabase offers a fully managed SQL database with features that make it suitable for applications needing a traditional relational database structure.

- **Features of Supabase:**
 - **PostgreSQL Database:** Supabase offers a fully managed PostgreSQL instance as a service. PostgreSQL is known for its reliability, ACID compliance, and powerful querying capabilities.
 - **Authentication:** Supabase provides built-in user authentication and authorization, which includes support for email/password, social logins, and more.
 - **Real-time Capabilities:** Supabase includes real-time subscriptions, allowing applications to subscribe to database changes and receive updates in real time.
 - **APIs and Querying:** Supabase auto-generates RESTful APIs for querying the database and can be accessed via simple HTTP requests, supporting CRUD operations.
 - **Storage:** Supabase offers object storage for managing files like images, videos, or documents in the cloud.
 - **Auto-scaling:** Supabase handles scaling of PostgreSQL databases automatically, ensuring that apps can handle increased loads.
- **Implementation:** Supabase provides a dashboard for easy management of the database and other services, along with client libraries for various programming languages. It offers an easy way for developers to work with relational databases, user authentication, and file storage, all while offering many of the features provided by Firebase.

Key Differences Between Firebase and Supabase

1. Database Type:

- Firebase primarily uses **NoSQL** databases (Firestore and Realtime Database), making it suitable for apps requiring flexible schemas and real-time data sync.
- Supabase uses **PostgreSQL**, a relational database, making it suitable for applications that require structured data and SQL querying.



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Year 2024 - 2025

SAP ID: 60019220028

2. Real-time Data:

- Firebase provides real-time capabilities with the Realtime Database and Firestore.
- Supabase also supports real-time data through its Postgres-based system, though it may not offer the same level of real-time data synchronization as Firebase.

3. Ease of Use:

- Firebase is known for its simplicity and seamless integration with Google's ecosystem, especially for mobile and web apps.
- Supabase, while simpler than traditional PostgreSQL setups, is more complex than Firebase due to the nature of SQL databases.

4. Pricing and Hosting:

- Firebase has a free tier but charges based on usage, including database reads/writes and storage.
- Supabase offers a free tier with limited usage, with pricing based on resource consumption (e.g., storage, API calls).



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT)

Academic Year 2024 - 2025

SAP ID: 60019220028

Output: 1.

Setup

python -m venv venv\Scripts\activate pip install firebase-admin supabase pyjwt requests python-dotenv

2. MOCK_DATA.json

```
1 [{"id":1,"first_name":"Astrid","last_name":"Posen","email":"aposen@yahoo.co.jp","gender":"Female","ip_address":"98.165.152.23"},
2 {"id":2,"first_name":"Bryan","last_name":"Scotchbourouge","email":"bscotchbourouge1@theatlantic.com","gender":"Male","ip_address":"185.63.90.92"},
3 {"id":3,"first_name":"Vern","last_name":"Bingell","email":"vbingell2@unicef.org","gender":"Male","ip_address":"105.30.242.67"},
4 {"id":4,"first_name":"Dulcinea","last_name":"MacAlaster","email":"dmacalaster3@cam.ac.uk","gender":"Female","ip_address":"151.61.87.39"},
5 {"id":5,"first_name":"Torre","last_name":"Fitzpatrick","email":"tfitzpatrick4@aboutads.info","gender":"Male","ip_address":"181.178.239.115"},
6 {"id":6,"first_name":"Rosco","last_name":"Prescott","email":"rprescott5@usgs.gov","gender":"Male","ip_address":"173.183.51.60"},
7 {"id":7,"first_name":"Shaylyn","last_name":"De Metz","email":"sdemetz6@yellowpages.com","gender":"Female","ip_address":"32.33.65.175"},
8 {"id":8,"first_name":"Sauveur","last_name":"Syder","email":"ssyder7@cdc.gov","gender":"Male","ip_address":"105.30.206.0"},
9 {"id":9,"first_name":"Kira","last_name":"Carlett","email":"kcarlett8@yahoo.com","gender":"Female","ip_address":"12.197.4.60"},
10 {"id":10,"first_name":"Georgianne","last_name":"Rubinsky","email":"grubinsky9@histats.com","gender":"Female","ip_address":"230.158.78.244"},
11 {"id":11,"first_name":"Caryl","last_name":"Kilby","email":"ckilbya@mozilla.org","gender":"Female","ip_address":"124.206.196.76"},
12 {"id":12,"first_name":"Terrance","last_name":"McCowan","email":"tmccowanb@discovery.com","gender":"Male","ip_address":"18.241.77.201"},
13 {"id":13,"first_name":"Dulcy","last_name":"Flindall","email":"dflindallc@auda.org.au","gender":"Female","ip_address":"101.215.72.2"},
14 {"id":14,"first_name":"Brittney","last_name":"Perrington","email":"bperringtond@nyu.edu","gender":"Female","ip_address":"100.187.208.101"},
15 {"id":15,"first_name":"Eannine","last_name":"Howsam","email":"jhowsame@msn.com","gender":"Female","ip_address":"158.251.165.25"},
16 {"id":16,"first_name":"Trixy","last_name":"Croxford","email":"tcroxfordf@tmall.com","gender":"Female","ip_address":"138.14.152.231"},
17 {"id":17,"first_name":"Angelico","last_name":"McCorkindale","email":"amccorkindaleg@smh.com.au","gender":"Male","ip_address":"59.45.166.75"},
18 {"id":18,"first_name":"Delphine","last_name":"Mielnik","email":"dmielnikh@umich.edu","gender":"Female","ip_address":"148.154.8.131"},
19 {"id":19,"first_name":"Syd","last_name":"Marchent","email":"smarchenti@theiglobeandmail.com","gender":"Male","ip_address":"59.8.184.135"},
20 {"id":20,"first_name":"Dew","last_name":"Kelston","email":"dkelstonj@skyrock.com","gender":"Male","ip_address":"167.130.56.250"},
21 {"id":21,"first_name":"Toiboid","last_name":"Joll","email":"tjollk@ucoz.ru","gender":"Male","ip_address":"36.73.147.160"},
22 {"id":22,"first_name":"Linus","last_name":"Levesley","email":"llevesleyl@uol.com.br","gender":"Male","ip_address":"20.77.255.223"},
23 {"id":23,"first_name":"Enrico","last_name":"Sheehy","email":"esheehym@ezinearticles.com","gender":"Male","ip_address":"127.53.178.100"},
24 {"id":24,"first_name":"Debera","last_name":"Ferrino","email":"dferrinon@columbia.edu","gender":"Female","ip_address":"140.25.102.192"},
25 {"id":25,"first_name":"Imelda","last_name":"Buddleigh","email":"ibuddleigho@aol.com","gender":"Female","ip_address":"52.203.217.54"},
26 {"id":26,"first_name":"Hali","last_name":"Rickeard","email":"hrickeardp@buzzfeed.com","gender":"Female","ip_address":"31.3.166.35"},
27 {"id":27,"first_name":"Malinde","last_name":"Cumberbatch","email":"mcumberbatchq@1688.com","gender":"Female","ip_address":"94.154.232.145"},
28 {"id":28,"first_name":"Sheffy","last_name":"Heigho","email":"sheighon@so-net.ne.jp","gender":"Male","ip_address":"84.90.115.123"},
29 {"id":29,"first_name":"Marijn","last_name":"Gergler","email":"mgerglers@so-net.ne.jp","gender":"Male","ip_address":"0.113.6.228"},
30 {"id":30,"first_name":"Arlana","last_name":"Diddams","email":"adiddamst@theguardian.com","gender":"Female","ip_address":"143.29.247.117"},
31 {"id":31,"first_name":"Vin","last_name":"Blunderfield","email":"vblunderfieldu@twitter.com","gender":"Female","ip_address":"40.167.83.176"},
32 {"id":32,"first_name":"Rianon","last_name":"Boniface","email":"rbonifacev@istockphoto.com","gender":"Female","ip_address":"213.83.197.122"},
33 {"id":33,"first_name":"Thaxter","last_name":"Menary","email":"tmenaryw@craigslist.org","gender":"Male","ip_address":"138.133.202.151"},
34 {"id":34,"first_name":"Major","last_name":"Simeonov","email":"msimeonovx@goo.ne.jp","gender":"Male","ip_address":"247.160.214.8"},
35 {"id":35,"first_name":"Ric","last_name":"Garroch","email":"rgarrochy@businessweek.com","gender":"Male","ip_address":"195.130.172.178"},
36 {"id":36,"first_name":"Carlee","last_name":"Aristide","email":"caristidez@reuters.com","gender":"Female","ip_address":"232.175.108.85"},
37 {"id":37,"first_name":"Odie","last_name":"Lorne","email":"olornei@technorati.com","gender":"Male","ip_address":"78.94.32.77"},
38 {"id":38,"first_name":"Vasilis","last_name":"Glason","email":"vglason11@newyorker.com","gender":"Male","ip_address":"26.160.24.110"},
39 {"id":39,"first_name":"Saunder","last_name":"Howsin","email":"showsini12@soundcloud.com","gender":"Male","ip_address":"73.206.209.180"},
40 {"id":40,"first_name":"Adrien","last_name":"Draude","email":"adraudel13@nsw.gov.au","gender":"Male","ip_address":"103.177.86.58"},
41 {"id":41,"first_name":"Ricky","last_name":"Mallettratt","email":"rmallettratt14@umn.edu","gender":"Male","ip_address":"120.198.254.0"},
42 {"id":42,"first_name":"Kathleen","last_name":"Poundsford","email":"kpoundsford15@senate.gov","gender":"Female","ip_address":"123.164.178.179"},
43 {"id":43,"first_name":"Darcy","last_name":"Gomersal","email":"dgomersal16@ebay.com","gender":"Genderfluid","ip_address":"9.84.211.247"},
44 {"id":44,"first_name":"Mirella","last_name":"Lanchbury","email":"mlanchbury17@salon.com","gender":"Female","ip_address":"96.237.228.11"},
45 {"id":45,"first_name":"Carlen","last_name":"Danslow","email":"cdanslow18@issuu.com","gender":"Female","ip_address":"49.187.199.233"},
46 {"id":46,"first_name":"Hayyim","last_name":"Gowler","email":"hgowler19@reverbnation.com","gender":"Male","ip_address":"237.222.109.20"},
47 {"id":47,"first_name":"Genevra","last_name":"Coolbear","email":"gcoolbear1a@reverbnation.com","gender":"Agender","ip_address":"242.89.83.165"},
48 {"id":48,"first_name":"Enos","last_name":"Insko","email":"einsko1b@tuttocitta.it","gender":"Male","ip_address":"132.89.147.236"},
49 {"id":49,"first_name":"Vovonna","last_name":"Blanket","email":"vblanket1c@jathis.com","gender":"Female","ip_address":"196.225.10.31"}]
```




Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT)

Academic Year 2024 - 2025

SAP ID: 60019220125

3.Firebase: NoSQL Database

Firestore.py

```
import time, csv, json import firebase_admin
from firebase_admin import credentials,
firestore

# Initialize Firebase cred = credentials.Certificate("test29051011-
firebase-adminsdk-fbsvc-
0d977f593f.json") # Replace with your actual
path firebase_admin.initialize_app(cred) db =
firestore.client()

# Load dataset from Test.json with
open("Test.json", "r") as f:
dataset = json.load(f)

def benchmark_firestore(data):
    results = []

    for item in data:
        doc_ref = db.collection("profiles").document(str(item["id"]))
        print(f"Inserting document: {item['id']}") # Add this line

        # CREATE t0 =
        time.time()
        doc_ref.set(item)
        create_lat = (time.time() - t0) * 1000

        # READ
        t0 = time.time()
        _ = doc_ref.get().to_dict()
        read_lat = (time.time() - t0) * 1000

        # UPDATE t0 =
        time.time()
        doc_ref.update({"last_name": item["last_name"] + "_updated"})
        update_lat = (time.time() - t0) * 1000 results.append((item["id"],
```

```
create_lat, read_lat, update_lat)) print("Benchmark completed.") #  
Confirm script finished
```



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT) Academic

Year 2024 - 2025

SAP ID: 60019220125

```
# Save CSV results
with open("firestore results.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "create ms", "read ms", "update ms"])
    writer.writerows(results)

# Run the benchmark
benchmark firestore(dataset)
```

CRUD operations

```
# CREATE
t0 = time.time()
doc_ref.set(item)
create_lat = (time.time() - t0) * 1000

# READ
t0 = time.time()
_ = doc_ref.get().to_dict()
read_lat = (time.time() - t0) * 1000

# UPDATE
t0 = time.time()
doc_ref.update({"last_name": item["last_name"] + "_updated"})
update_lat = (time.time() - t0) * 1000
```

```
PS E:\Migration\Temp\CCL> python .\Firestore.py
Inserting document: 1
Inserting document: 2
Inserting document: 3
Inserting document: 4
Inserting document: 5
Inserting document: 6
Inserting document: 7
Inserting document: 8
Inserting document: 9
Inserting document: 10
Inserting document: 11
Inserting document: 12
Inserting document: 13
Inserting document: 14
Inserting document: 15
Inserting document: 16
Inserting document: 17
Inserting document: 18
Inserting document: 19
Inserting document: 20
Benchmark completed.
```



Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

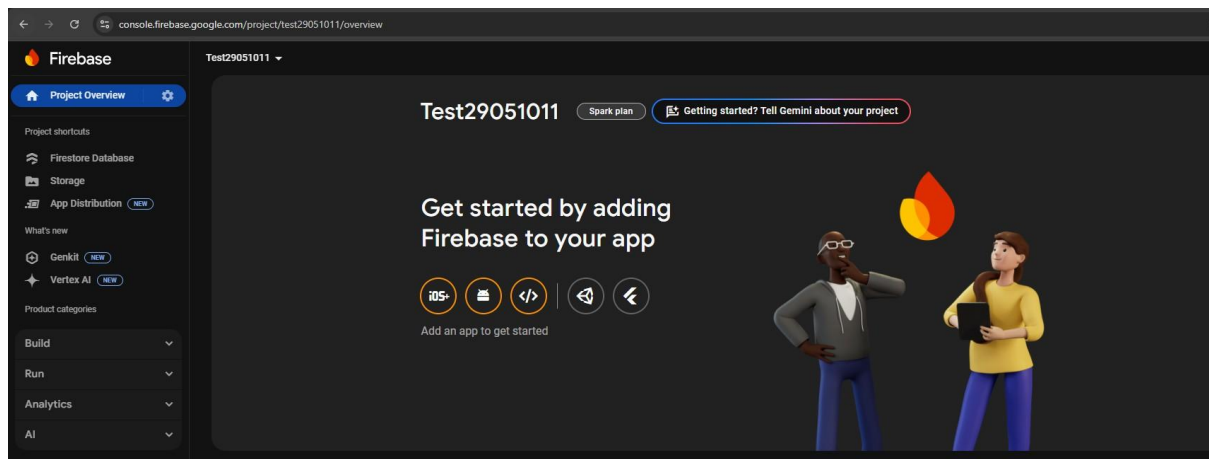


Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT)

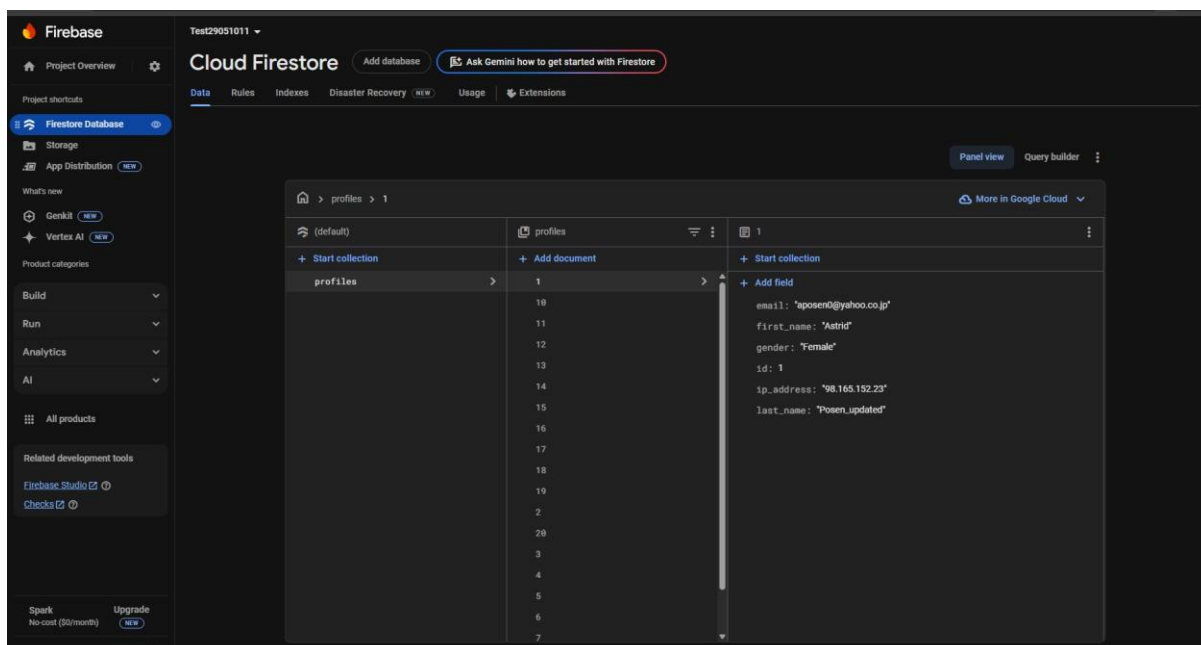
Academic Year 2024 - 2025

SAP ID: 60019220125

Firestore Studio



Cloud Firestore:





Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT) Academic

Year 2024 - 2025

SAP ID: 60019220125

4. Supabase : SQL Database

Supabase.py

```
import time import csv import json import
os from dotenv import load_dotenv from
supabase import create_client, Client

# Load credentials
load_dotenv()
SUPABASE_URL = os.getenv("SUPABASE_URL") SUPABASE_KEY =
os.getenv("SUPABASE_KEY") supabase: Client =
create_client(SUPABASE_URL, SUPABASE_KEY)

# Load dataset with
open("MOCK_DATA.json", "r") as f:
    dataset = json.load(f)

# Optional: test a single record before
benchmarking print("🔍🔍 Testing insert with dummy
data...) try:
    test_result = supabase.table("profiles").insert({
        "id": 9999,
        "first_name": "Test",
        "last_name": "User",
        "email": "testuser@example.com",
        "gender": "Other",
        "ip_address": "127.0.0.1"
    }).execute() print("✅ Test insert
successful:", test_result)
except Exception as e:
    print("❌ Test insert failed:", e)

# Main Benchmarking Function def
benchmark_supabase(data):
    results = []

    for item in data:
        print(f"\n📄📄 Inserting document ID: {item['id']}")
        try:
```

CREATE



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT) Academic

```
Year 2024 - 2025  SAP ID: 60019220028  t0 = time.time()  insert_response =
supabase.table("profiles").insert({
    "id": item["id"],
    "first_name": item["first_name"],
    "last_name": item["last_name"],
    "email": item["email"],
    "gender": item["gender"],
    "ip_address": item["ip_address"]
}).execute()  create_lat =
(time.time() - t0) * 1000  print("✅
Inserted successfully.")

# READ  t0 = time.time()
supabase.table("profiles").select("*").eq("id",
item["id"]).single().execute()  read_lat =
(time.time() - t0) * 1000

# UPDATE  t0 = time.time()
supabase.table("profiles").update({
    "last_name": item["last_name"] + "_updated"
}).eq("id", item["id"]).execute()  update_lat = (time.time()
- t0) * 1000  results.append((item["id"], create_lat,
read_lat, update_lat))

except Exception as e:
    print(f"❌ Error processing ID {item['id']}: {e}")

print("\n✅ Benchmark completed.")

# Save results to CSV with
open("supabase_results.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "create_ms", "read_ms", "update_ms"])
    writer.writerows(results)
print("📄📄 Results saved to supabase_results.csv")

# Run benchmark benchmark_supabase(dataset)
```



Department of Computer Science & Engineering (I.O.T. & Cybersecurity with BCT)

Academic Year 2024 - 2025

SAP ID: 60019220125

Inserting Document:

```
PS E:\Migration\Temp\CCL> python .\Supabase.py
Testing insert with dummy data...
Test insert successful: data=[{'id': 9999, 'first_name': 'Test', 'last_name': 'User', 'email': 'testuser@example.com', 'gender': 'Other', 'ip_address': '127.0.0.1'}]
count=None

Inserting document ID: 1
Inserted successfully.

Inserting document ID: 2
Inserted successfully.

Inserting document ID: 3
Inserted successfully.

Inserting document ID: 4
Inserted successfully.

Inserting document ID: 5
Inserted successfully.

Inserting document ID: 6
Inserted successfully.

Inserting document ID: 7
Inserted successfully.

Inserting document ID: 8
Inserted successfully.
```

DBaaS:

id	first_name	last_name	email	gender	ip_address
1	Arly	Absalom_updated	aabsalom0@sciencedaily.com	Female	242.
2	Kandy	Gannicott_updated	kgannicott1@netvibes.com	Female	146.
3	Hewet	Cattlemull_updated	hcattlemull2@ca.gov	Male	78.9
4	Vail	Casterton_updated	vcasterton3@apple.com	Male	122.
5	Allister	Dilston_updated	adilston4@pagesperso-orange.fr	Male	87.4
6	Huntington	O'Dennehy_updated	hodenney5@prlog.org	Male	88.8
7	Aylmer	Goodwill_updated	agoodwill6@latimes.com	Male	73.2
8	Corine	Thirlaway_updated	cthirlaway7@deliciousdays.com	Female	174.
9	Deanne	Soitoux_updated	dsoitoux8@wunderground.com	Female	136.
10	Ericha	Arldige_updated	earldige9@ucsd.edu	Female	17.16
11	Waylan	Breakwell_updated	wbreakwelle@house.gov	Male	68.2
12	Chico	Celand_updated	ccelandb@google.com.au	Male	94.1
13	Locke	Vella_updated	lvellac@redcross.org	Male	0.75
14	Sosanna	Darey_updated	sdareyd@bbc.co.uk	Female	218.
15	Rocky	Mougin_updated	rmougine@home.pl	Male	115.4
16	Consolata	Marconi_updated	cmarconif@mlb.com	Female	188.

Conclusion: In conclusion, DBaaS provides managed, scalable database solutions for applications, with Firebase catering to NoSQL needs and real-time data, and Supabase offering a powerful SQL-based solution for relational data. Both platforms provide ease of use, but Firebase is more focused on flexibility and real-time synchronization, while Supabase is ideal for structured, relational data with SQL querying.